

A Novel Framework for Accelerating Core Loading Pattern Optimization with Deep Learning Surrogate Models

Daniel Williams*, Paul Seurin[†], Koroush Shirvan[†]

**Department of Computer Science and Electrical Engineering, University of Maryland, Baltimore County,
1000 Hilltop Circle, Baltimore, MD 20852*

*[†] Department of Nuclear Science and Engineering, Massachusetts Institute of Technology
77 Massachusetts Avenue, Cambridge MA, 02139
dwillia2@umbc.edu, paseurin@mit.edu, kshirvan@mit.edu*

INTRODUCTION

Finding an optimal loading pattern for the reactor core of Nuclear Power Plants (NPPs) has challenged nuclear fuel designers since the 1960s. The core loading pattern optimization itself being a complex multi-objective combinatorial optimization problem [1], designers have relied upon engineering experience when choosing critical attributes (fuel placement, enrichment, neutron poisons, etc.) to reach satisfying levels of safety, efficiency, and economy of the reactor core [2].

Advancements in the Machine Learning (ML) field during the last decade, in particular in the area of Reinforcement Learning (RL) have shown promise to solve this problem, but not without challenges of their own. Recent work has produced an RL-based framework to tackle this optimization through developed AI agents designed to learn to build optimal nuclear reactor core designs. There, with the help of a nuclear safety tool with licensing-grade accuracy, the agents, represented by deep learning architectures, learned to construct optimal patterns by searching to minimize an objective function, which encapsulated economic, safety, and operational constraints [2]. However, the computing cost incurred by the full physics simulation code during agent training prevents the agents to collect a large number of samples [3]. To overcome this bottleneck, several authors replace the computationally intensive evaluation of the physics code with fast surrogate models [4, 5, 3].

While the work in [2] has revolved around tuning RL hyper-parameters to find patterns rapidly rather than leveraging surrogates, it is known that to perform better, deep learning models would benefit from a large amount of data. Nevertheless, classical works using surrogates usually leveraging simple models, are limited by the Figure of Merits (FOM)'s trained on (e.g., the relative power distribution in [4]), and the models need to be dynamically retrained to prevent losses in accuracy [5]. Moreover, some simple models could be utilized to predict some FOM (e.g., the cycle length [4]) but authors are still utilizing complicating architectures.

Advances in deep models offer promise to improve the accuracy of the surrogates for more complex core characteristics while being fast to evaluate. However, the training time becomes an obstacle even for simpler problems [6]. Recent work in [1] is leveraging these advances to speed up the collection of samples for optimization using different heuristics such as Particle Swarm Optimization (PSO). However, the training time of the model with pre-generated samples amounted to 100h for each FOM.

There is an apparent trade-off between prediction accuracy, sample collection and training times, and model rele-

vance which these authors failed to incorporate efficiently. Our work attempts to embed these trade-offs in a novel framework for leveraging deep surrogate modeling within a representative core loading pattern optimization problem.

Here, we developed a Deep Convolutional Neural Network (CNN) which predicts the maximum boron concentration, from core loading pattern samples produced by similar agents than in [2]. To limit the number of samples collected for training, while ensuring high accuracy, we also develop a class-imbalance aware loss function and post-model error correction techniques with Extreme Gradient Boosted Trees (XGBoost). While using this model, we found that collecting 30,000 samples is sufficient for training a viable deep-learning surrogate model that achieves an R^2 score of 0.986 on 40,000 unseen samples found during the RL optimization. Considering the low training time of the CNN and XGBoost models (about 15 min with 100 epochs), this represents a potential gain of 156h compared to using the full physics code. This could open the door to much more detailed core optimization which was hindered by the computational cost of classical solvers.

SURROGATE MODELING OPTIMIZATION

Surrogate modeling has been a key approach in various works where classical methods are too computationally costly to be deployed. For instance, Artificial Neural Networks (ANN)s and 2D computations have been successfully applied in [4] and [5], respectively for PWR optimization. There, the surrogate helped screen out poor quality patterns instead of evaluating every solution with the slow physics code, while a Genetic Algorithm (GA) and Simulated Annealing (SA) drove the optimization, respectively. [3] trained a Gaussian Process (GP) to accelerate the evaluation of candidates during a multi-objective GA-based optimization of the ABTR, a small Sodium Fast cooled reactor. There, a GP fully replaces the very costly physics solver (~13 min per evaluation) reducing the optimization run-time by an order of magnitude. These works suffer from some critical limitations. For one, 2D computations may remain slow and not significantly accelerate the number of samples collected compared to a surrogate. [4] surrogates were based on a 2D core calculation which already embeds errors compared to 3D simulations and the fuel diversity (i.e., the decision variables) was limited to fixed enrichment, which is not very representative of real-world application. Additionally, only 4000 candidates' patterns were generated which questions the diversity and the necessary extrapolation needed when a bigger search space is available.

Sometimes, the problem can be simpler than a representa-

tive full core, which does not ensure scalability. In [3], only 7 inputs are utilized for a small core where fuel assemblies are radially homogenized. Nevertheless, for the problem posed in this work, many more assemblies are involved and have radial heterogeneity, and unfortunately, GP does not scale well for high dimensional data.

[6] proposed a study with CNN-based models commenting on the accuracy, computing-, and inference-time penalties in scaling up from single PWR fuel assembly to multi-region up to 4x4 for the prediction of normalized pin power. Nevertheless, even if it is the ultimate goal, their implementation was not integrated into an optimization. The number of trainable parameters when scaling up can become unreasonably large and they even argue that they do not envision it to replace the full physics code, which contrasts with much of the literature arguing about using a surrogate model as a sole evaluator. Furthermore, the training amounted to a couple of hours on a single GPU, which could already be considered significant for a full-core PWR. [1] offers an interesting example. They scale up to real core design with a deep model by pre-generating 100,000 training samples. But as stated earlier, the training time there was too prohibitive. Moreover, the data collection time is not provided, they are not re-training, and the accuracy of the models on new data unveiled during the optimization was very poor. When an optimization is conducted on a new core design it is unclear if the models will extrapolate and therefore, they may need to spend another prohibitive time collecting data and training their model.

Lastly, it is unclear if the methods used could be outperformed by others, as no thorough analysis of the model with statistical tests has been performed. Typically, [4] has shown that a linear correlation for predicting the cycle length is already enough. While [3] performs a sensitivity analysis based on the Pearson coefficient to relate interesting inputs to desired outputs, it is known that most of the existing model can capture non-linearity sometimes to a smaller cost than a GP. As a result, we argue that to scale up to real-world applications we need:

1. To achieve high accuracy on a representative core design.
2. With the lowest computing cost possible, embedding the data collection time and ensuring accuracy during training and at test time (i.e., when predicting unseen samples during optimization).
3. To achieve so, perform a thorough model comparison with physics knowledge to have the most adapted model for each figure of merits predicted.

DEEP LEARNING MODEL UTILIZED IN THIS WORK

Recent research in PWR surrogate modeling has employed various supervised learning techniques such as Linear Regression, ensemble techniques like random forests (RF) and extreme gradient boosted trees (XGB), and kernel methods such as Gaussian processes regression (GP), and support vector regression (SVR) [7]. However, a key problem remains that often these models do not encode the high dimensionality typically present within nuclear loading pattern data into the

models well, leading to estimators that do not generalize well outside of their training data.

We chose to specifically focus our pursuit in the field of deep learning, due to the recent track record of success shown in a far-ranging set of problems across scientific disciplines. CNNs are learning models that operate on multidimensional array data such as RGB images and learn to assign importance to the groups of pixels that help it succeed in a learning task, pruning the irrelevant connections and reducing the number of trainable parameters in the process. This differs from classical ANNs which effectively treat neighboring pixels the same as those distant. When dealing with 2D core loading pattern data, neighboring fuel assemblies ought to form stronger relationships than those far away, wherein the physical interactions between neighboring groups of assemblies produce the large-scale physics seen across the reactor. The comparisons of the CNN against classical machine learning models will be given in this work in the *RESULTS* section.

DESCRIPTION OF THE NOVEL FRAMEWORK

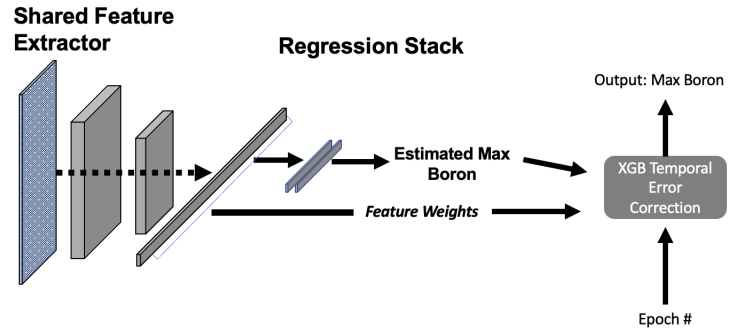


Fig. 1. Three-Stage Framework Architecture

TABLE I. Model inputs. IFBA and WABA stand for Integral Fuel Burnable Absorber and Wet Annular Burnable Absorber, respectively. The exposures are considered at Beginning of Cycle (BOC).

Design Parameter Inputs	Data Type	Allowed Range
Fuel Placement	Integer	0.0-70.0
Enrichment	Float	4.0-4.95
IFBA	Integer	128.0-156.0
WABA	Integer	0.0-24.0
Average Exposure	Float	0.0-58.449
Pin Peak Exposure	Float	0.0-62.996
k_{∞}	Float	0.0-1.144

The framework described in figure 1, is composed of three modules that interact with [2]’s RL-based optimization. After gathering enough data for the surrogate model training, the framework receives a prospective core embedding produced by the agent for evaluation. The embedding is constructed with a 15 x 15 image for 7 different features, which we considered

based on our knowledge of the physics of a nuclear reactor. For each loading pattern, we stack the features in 7 channels. The 7x15x15 input embedding of the reactor core is passed to the surrogate model, wherein the CNN estimates the scalar max boron. Table I further describes the CNN model inputs with their types and ranges. As we can see the ranges differ significantly between the inputs, therefore we normalized the input, which is a common strategy for the models to treat each feature uniformly improving the accuracy of the prediction in the process.

Unlike [6], our CNN architecture is composed of two sequentially connected blocks of convolutional layers that act as feature extractors for the core pattern input. These learned extracted features feed into a regression stack composed of two fully connected layers each followed by the output layer returning the predicted value of the FOM of interest. Here, our main focus is the maximum boron concentration. Ideally, after the prediction by the CNN, the other FOMs are predicted and passed back to [2] agent as feedback. However, we realized a glaring imbalance of data between the early patterns found and the latest as shown in figure 2, where the early patterns have very high $F_{\Delta h}$ compared to later ones.

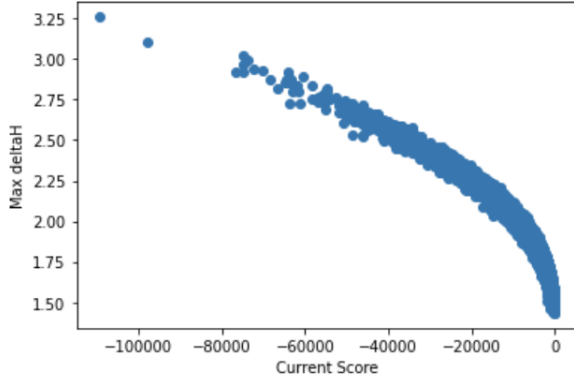


Fig. 2. Distribution of the $F_{\Delta h}$ based on when the corresponding patterns appear in the optimization.

We hypothesize that, as the algorithm converges to high-quality area of the search space, the pattern differs significantly from the earlier ones. Hence, models pre-trained with very few or random samples will fail to accurately predict the FOMs of the later (optimized) cores. To tackle this issue, first, we decided to derive a custom loss function that accounts for the imbalance in the training data, increasing the magnitude of the loss for outliers in the dataset. The formulation of the custom loss weighting scheme is given in equation 1:

$$L(x_i) = \begin{cases} (1 - \log(\frac{x_i}{\bar{x}})) & \text{if } x_i \leq \bar{x} - \sigma \\ (1 + \log(\frac{x_i}{\bar{x}})) & \text{if } x_i \geq \bar{x} + \sigma \\ 1 & \text{otherwise} \end{cases} \quad (1)$$

Where x_i is an individual core loading sample, $\bar{x}$ is the computed sample mean of the training data, and σ is the computed sample standard deviation for the training data. Secondly, to improve the performance even more with minimum cost, we will experiment with an error correction computed via XGBoost, a Machine Learning model that combines both fast

training and inference times. In this experiment, the lower dimensional learned prediction of the CNN is fed to XGBoost, which predicts the boron concentration. Lastly, to ensure that the trained model will generalize to unseen loading patterns, we will feed the dataset as they were being generated by the RL agents. Then, we will train the model at different times based on the data already collected and test it on the remainder of the data. In this regard, we can gauge how many samples must be collected prior to fully replace the computer program.

RESULTS

Each model is trained on one node with 2 Intel Xeon 2.1 GHz CPU with 128 GB RAM and a Tesla K20m GPU.

Performance of CNN

First, the CNN is compared in predicting Boron against classical machine learning models namely Linear Regression, non-linear kernel ridge, XGBoost, and ANN. For all but the CNN, the models are unable to handle 2D data, so the average of the 7 inputs from 2D observations are computed and used.

TABLE II. Comparison of the different models based on a (5) K-Fold Cross Validation. The best model appears in bold.

Models	RMSE	R^2	Max Error	Mean Error
Linear Regression	22.983	0.886	130.325	17.112
Kernel Ridge	25.188	0.882	161.844	16.457
XGBoost	20.085	0.913	153.784	15.260
ANN	22.924	0.88	130.325	17.112
CNN	5.510	0.993	40.093	4.266
CNN with Custom Loss	5.392	0.994	40.835	4.079
Error Correction on Best Model	5.065	0.994	28.408	3.820

As we can observe from Table II, all instances of the CNN (regular, with custom loss, and with XGBoost error correction) present a lower RMSE and higher R^2 than any other classical model. The max and mean absolute errors are also given. The former is interesting because, based on designer inputs, we hope that the max error on boron is below 20 ppm. Otherwise, we cannot replace the code with the surrogate. In this aspect, XGBoost is interestingly the second-best performing model behind the CNN, even outperforming the ANN. This is a motivation for leveraging it as a temporal error corrector which leads to tangible decreases in max error (last row of Table II). Lastly, a value of 0.8 for R^2 is sometimes considered as an acceptable model performance. Nevertheless, here we see with the max and mean error that even with 0.88, we should not accept linear or kernel ridge regression.

Deployment of the CNN within an optimization framework

To remain deployable for real-world loading pattern optimization, the number of training samples and the time allocated to train the surrogate must remain feasible to computational and time constraints within these settings (which is about 1 to 3 days [2]). To address this, we decided to train the model online and assess when it can replace the physics code. Therefore, the performance of our CNN model is tested on different sizes of training batches. The errors will be tested on the remainder of the data. We also provide the potential training time saved. We evaluated a 0.042 s inference time for our model against 10 s inference time for the physics code. Then we multiply the difference by the number of samples evaluated solely by our model. Moreover, for the same reasons highlighted in the last section, we also count the number of miss-predicted boron by our model.

Table III shows that 30,000 represents an interesting trade-off. The R^2 and mean errors do not vary significantly by adding more samples while the potential training time saved is substantial (156hr or 6.5 days). Moreover, only 66 patterns are miss-predicted, which remains very low with regards to the 40,000 unseen patterns it tried to predict for.

TABLE III. Pre-training of our CNN with custom loss on Subsets of Size k. The training time is 5 minutes for the 10,000 samples case and increases linearly with the number of samples. The total number of data collected is 70,000.

# Samples	RMSE	R^2	Max Error	Mean Error	# Errors above 20ppm	Time Saved (hrs.)
10,000	10.875	0.971	45.579	8.690	1163	211
20,000	7.255	0.986	32.351	5.759	132	184
30,000	6.377	0.987	36.414	5.044	66	156
40,000	5.213	0.990	28.615	4.113	11	128
50,000	5.589	0.987	30.621	4.484	10	100
60,000	4.617	0.989	20.735	3.696	1	73

CONCLUSION

Advances in deep learning offer promise in increasing the accuracy of the classical machine learning models to the cost of extra training time. By accounting for the imbalance of data incurred by the temporality of the samples generated for the model, we show that our framework can be leveraged to train high-performance surrogates ensuring low training time (~ 15 min over the full optimization), high accuracy (66 out of 40,000 boron miss-predicted), and substantial computing cost savings on a real-world nuclear reactor core (156hr or 6.5 days).

REFERENCES

1. Z. LI, J. HUANG, J. WANG, and M. DING, "Comparative study of meta-heuristic algorithms for reactor fuel reloading optimization based on the developed BP-ANN calculation method." *Annals of Nuclear Energy*, **165**, pp

- 108685 (2022).
2. P. SEURIN and K. SHIRVAN, "PWR Loading Pattern Optimization with Reinforcement Learning," in "International Conference on Physics of Reactors 2022 (PHYSOR 2022)," Pittsburgh, PA, May 15-20 (2022).
3. K. ZENG, N. E. STAUFF, J. HOU, and T. KIM, "Development of multi-objective core optimization framework and application to sodium-cooled fast test reactors," *Progress in Nuclear Energy*, **120**, pp 103184 (2020).
4. A. YAMAMOTO, "Application of Neural Network for Loading Pattern Screening of In-Core Optimization Calculations," *Nuclear Technology*, **144**, 1, 63–75 (2003).
5. T. K. PARK, H. G. JOO, and C. H. KIM, "Multiobjective Loading Pattern Optimization by Simulated Annealing Employing Discontinuous Penalty Function and Screening Technique," *Nuclear Science and Engineering*, **162**, 134–147 (2009).
6. F. SHRIVER and J. WATSON, "Scaling deep learning for whole-core reactor simulation," *Progress in Nuclear Energy*, **146**, pp 104134 (2022).
7. C. YIFENG, J. YURKO, P. SEURIN, and K. SHIRVAN, "Machine Learning-Assisted Surrogate Construction for Full-Core Fuel Performance Analysis," *Annals of Nuclear Energy*, **168**, pp 108905 (2022).